

Datenbanken

Lernskript zur Vorbereitung auf die mündliche Prüfung

Stand: 11.6.2026

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Einführung & Grundbegriffe	4
Warum Datenbanken?	4
Typische mündliche Fragen.....	4
2. DBMS-Architektur & Datenunabhängigkeit.....	5
Datenunabhängigkeit.....	5
Wichtige DBMS-Komponenten	5
3. Datenmodelle im Überblick	6
Relational vs. objektorientiert.....	6
4. Relationales Datenmodell	7
Relationale Algebra	7
Typische mündliche Fragen.....	7
5. SQL-Grundlagen.....	8
CREATE TABLE und Constraints.....	8
SELECT-Reihenfolge	8
Views	9
6. Datenbankentwurf mit ER-Modell	9
Anforderungen an den Entwurfsprozess	10
Vorgehen aus einer Aufgabenstellung.....	10
Kardinalitäten und Notationen	11
7. Normalisierung.....	11
Anomalien.....	11
Funktionale Abhängigkeit	11
Die ersten drei Normalformen.....	12
Denormalisierung.....	12
8. Konsistenz, Integrität & Transaktionen	12
Transaktionen & ACID	13
Transaktionstypen	13
9. Recovery-Verfahren	13
Logging	13
Rollback und Rollforward	14
Grober Recovery-Ablauf	14
10. Sperren, Isolation & Synchronisation	14
Typische Nebenläufigkeitsprobleme	14
Sperren	14
Isolationsebenen nach SQL.....	15
Deadlocks und optimistische Synchronisation.....	15
11. Datenspeicherung & Pufferverwaltung	15
Speicherhierarchie	15
DBMS-Puffer.....	15
Speicherorganisation	16

12. Indexe, B-Bäume & Zugriffspfade.....	16
B-Baum/B+-Baum – wichtige Idee.....	17
Beispiel aus der Übung.....	18
Typische mündliche Fragen.....	18
13. Anfrageverarbeitung & Optimierung	18
Grobablauf.....	18
Regelbasierte vs. kostenbasierte Optimierung	18
Einflussmöglichkeiten	19
14. Benutzerverwaltung & Rechte	19
15. Prüfungsaufgaben aus den Unterlagen	19
Aufgabe: CREATE TABLE mit Constraints	19
Aufgabe: Hochschule als ER-Modell	20
16. Kurz-Wiederholung für die mündliche Prüfung	20

1. Einführung & Grundbegriffe

Früher wurden Anwendungen häufig als Insellösungen gebaut: Jede Anwendung verwaltete ihre eigenen Daten. Dadurch waren Daten und Programme eng gekoppelt. Bei Änderungen an der Datenstruktur mussten oft mehrere Programme angepasst werden.

Begriff	Bedeutung
Datenbank	Menge weitgehend redundanzfreier Daten, die unter Steuerung eines DBMS gespeichert, verwaltet und manipuliert werden.
DBMS	Datenbank-Management-System: Programme, die als einzige direkt Operationen auf den Daten ausführen dürfen.
Datenbanksystem	Datenbank + DBMS zusammen.
Modell	Zielgerichtetes Abbild eines Ausschnitts der realen Welt.
Schema	Formalisierte Darstellung des Modells, z. B. grafisch oder als Tabellenstruktur.
Metadaten	Daten über Daten: Tabellen, Attribute, Typen, Schlüssel, Rechte, Sichten, Speicherstrukturen.

Warum Datenbanken?

- Redundanzminderung: Daten sollen möglichst nicht mehrfach und widersprüchlich gespeichert werden.
- Konsistenz und Integrität: Das DBMS kann Regeln prüfen und ungültige Zustände verhindern.
- Datenunabhängigkeit: Programme sollen nicht von physischen Speicherdetails oder jeder logischen Erweiterung abhängig sein.
- Zentrale Verwaltung: Sicherheit, Rechte, Sicherung, Wiederherstellung und Mehrbenutzerbetrieb werden gebündelt.
- Mehrfachnutzung: Viele Anwendungen greifen auf denselben Datenbestand zu, aber über kontrollierte Schnittstellen.



Wenn du nach dem Sinn von Datenbanken gefragt wirst, antworte nicht nur 'zum Speichern von Daten'. Wichtig sind: Redundanzen vermeiden, Konsistenz sichern, Daten unabhängig von Programmen verwalten und mehreren Nutzern kontrollierten Zugriff ermöglichen.

Typische mündliche Fragen

- **Frage:** Was ist der Unterschied zwischen Datenbank, DBMS und Datenbanksystem?
- **Antwortidee:** Datenbank = Datenbestand, DBMS = Verwaltungssoftware, Datenbanksystem = beides zusammen.
- **Frage:** Warum sind Insellösungen problematisch?
- **Antwortidee:** Weil gleiche Daten mehrfach liegen können, Änderungen schwer synchronisierbar sind und Programme an Datenstrukturen hängen.

2. DBMS-Architektur & Datenunabhängigkeit

Das ANSI-SPARC-Modell trennt ein Datenbanksystem in drei Ebenen. Dadurch werden verschiedene Sichten auf dieselben Daten möglich und Änderungen auf einer Ebene sollen möglichst wenig Auswirkungen auf andere Ebenen haben.

Ebene	Aufgabe	Merke
Externe Ebene	Sicht einzelner Anwendungen oder Benutzergruppen.	Viele externe Schemata möglich.
Konzeptuelle Ebene	Logische Gesamtstruktur der Daten, Beziehungen und Integritätsregeln.	Anwendungsneutraler Gesamtentwurf.
Interne Ebene	Physische Speicherung auf externen Medien, Indexe, Blöcke, Speicherstrukturen.	Hardware- und performancebezogen.

Datenunabhängigkeit

Art	Bedeutung	Beispiel
Physische Datenunabhängigkeit	Programme bleiben unverändert, obwohl sich die physische Speicherung ändert.	Index hinzufügen, Partitionierung ändern, Speicherort verschieben.
Logische Datenunabhängigkeit	Externe Sichten/Programme bleiben unverändert, obwohl die logische Gesamtstruktur erweitert wird.	Neue Attribute für Einkauf ergänzen, Lager-Anwendung bleibt gleich.

Wichtige DBMS-Komponenten

Komponente	Aufgabe
Datensystem	Setzt logische Anfragen und Operationen um.
Zugriffssystem	Organisiert Zugriffspfade, Indexe und Suchstrategien.
Speichersystem	Verwaltet physische Datenblöcke, Puffer und Datenträgerzugriffe.
Metadatenverwaltung	Verwaltet Kataloginformationen über Schema, Rechte, Sichten und Speicherstrukturen.
Transaktionsverwaltung	Sichert ACID-Eigenschaften, Sperren, Logging und Recovery.

Externe Ebene ↔ logische Datenunabhängigkeit ↔ konzeptuelle Ebene ↔ physische Datenunabhängigkeit ↔ interne Ebene. Diese Kette eignet sich sehr gut als mündliche Antwort.

3. Datenmodelle im Überblick

Ein Datenmodell legt fest, mit welchen Strukturtypen Objekte, Beziehungen und Operationen beschrieben werden. In der Prüfung ist vor allem wichtig, die Modelle voneinander abzugrenzen und das relationale Modell sicher zu erklären.

Modell	Grundidee	Stärken / Schwächen
Hierarchisches Modell	Daten sind baumartig organisiert; ein Datensatz hat typischerweise genau einen Vorgänger.	Einfach bei klaren Hierarchien, aber unflexibel bei M:N-Beziehungen.
Netzwerkmodell	Datensätze können mehrere Beziehungen und Vorgänger haben.	Flexibler als hierarchisch, aber komplexere Navigation.
Relationales Modell	Daten werden als Tabellen/Relationen dargestellt; Beziehungen über Schlüssel und Fremdschlüssel.	Standard in vielen DBMS; mengenorientiert und gut formalisiert.
Objektorientiertes Modell	Datenbasis besteht aus Objekten mit Identität, Klassen, Methoden und komplexen Strukturen.	Gut für komplexe Objekte, aber weniger verbreitet und teils geringere Datenunabhängigkeit.
Objektrelationales Modell	Relationales Modell wird um objektorientierte Konzepte/Typen erweitert.	Kombiniert relationale Stabilität mit komplexeren Datentypen.

Relational vs. objektorientiert

- **Relational:** mengenorientiert, Tabellen, Joins, SQL, Beziehungen über Werte.
- **Objektorientiert:** objektorientierte Denkweise mit Objektidentität, Klassen, Vererbung, Kapselung und Methoden.
- **Objektrelational:** relationales Fundament, aber mit Erweiterungen wie strukturierten Typen, geschachtelten Tabellen oder benutzerdefinierten Typen.



Wenn nach Vor- und Nachteilen gefragt wird: Relationale Systeme sind gut standardisiert und datenunabhängig. Objektorientierte Systeme können komplexe Objekte natürlicher abbilden, koppeln Struktur und Operationen aber oft stärker.

4. Relationales Datenmodell

Das relationale Datenmodell stellt Daten als Relationen dar. Praktisch entspricht eine Relation meistens einer Tabelle. Eine Zeile heißt Tupel, eine Spalte heißt Attribut und der Wertebereich eines Attributs heißt Domäne.

Begriff	Bedeutung	Beispiel
Relation / Tabelle	Menge von Tupeln mit gleicher Struktur.	Student
Tupel / Zeile	Eine konkrete Wertekombination.	(G240039TI, Noah, Schmitz)
Attribut / Spalte	Eigenschaft eines Objekts.	Matrikelnummer, Name
Domäne	Zulässiger Wertebereich eines Attributs.	PLZ: fünfstellige Zeichenkette
Primärschlüssel	Eindeutige Identifikation eines Tupels.	Matrikelnummer
Fremdschlüssel	Attribut, das auf den Schlüssel einer anderen Relation verweist.	SBID in Studienrichtung

Relationale Algebra

Operation	Ergebnis	SQL-Entsprechung
Selektion	Auswahl von Zeilen, die eine Bedingung erfüllen.	WHERE
Projektion	Auswahl bestimmter Spalten.	SELECT spalte1, spalte2
Join	Verknüpfung zweier Relationen über vergleichbare Attribute.	JOIN ... ON ...
Equi-Join	Join mit Gleichheitsbedingung.	A.id = B.a_id
Natural Join	Gemeinsame Attribute werden nur einmal übernommen.	NATURAL JOIN
Outer Join	Nimmt auch Tupel ohne Join-Partner auf und füllt mit NULL auf.	LEFT/RIGHT/FULL OUTER JOIN



Selektion = Zeilen filtern. Projektion = Spalten auswählen. Join = Tabellen logisch zusammenführen.

Typische mündliche Fragen

- **Frage:** Was ist eine Domäne?
- **Antwortidee:** Der Wertebereich, aus dem ein Attribut seine Werte beziehen darf.
- **Frage:** Wie werden Beziehungen im relationalen Modell dargestellt?
- **Antwortidee:** Nicht durch Zeiger im logischen Modell, sondern durch passende Attributwerte: Primärschlüssel und Fremdschlüssel.

5. SQL-Grundlagen

SQL JOIN-Typen (Venn-Diagramm)

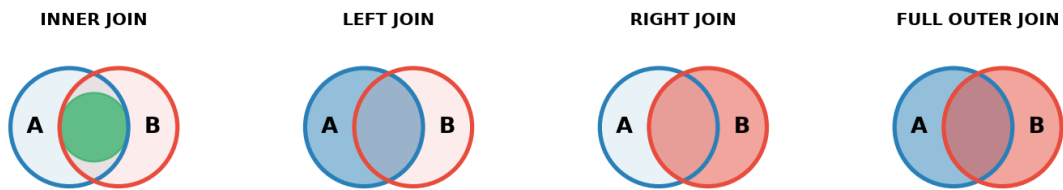


Abb.: SQL JOIN-Typen

SQL ist die wichtigste Schnittstelle zu relationalen Datenbanken. Der Sprachumfang unterscheidet sich je nach DBMS, aber die Grundgruppen sind stabil: Datendefinition, Datenmanipulation, Rechteverwaltung und Transaktionssteuerung.

Gruppe	Typische Befehle	Zweck
DDL	CREATE, ALTER, DROP	Strukturen anlegen, ändern, löschen.
DML	SELECT, INSERT, UPDATE, DELETE	Daten abfragen und verändern.
DCL	GRANT, REVOKE	Rechte vergeben oder entziehen.
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transaktionen abschließen oder zurücksetzen.

CREATE TABLE und Constraints

Constraint	Bedeutung
NOT NULL	Mussfeld: In jeder Zeile muss ein Wert vorhanden sein.
UNIQUE	Werte oder Wertkombinationen müssen eindeutig sein.
PRIMARY KEY	Primärschlüssel: eindeutig und nicht NULL.
FOREIGN KEY	Fremdschlüssel: verweist auf Schlüssel einer anderen Tabelle.
CHECK	Einfache Integritätsregel, z. B. CHECK (mietpreis > 0).

```
CREATE TABLE kunde (  
  kdnr      INTEGER PRIMARY KEY,  
  name      VARCHAR(30) NOT NULL,  
  vorname   VARCHAR(30),  
  adresse   VARCHAR(60)  
);  
  
CREATE TABLE miete (  
  mvnr      INTEGER PRIMARY KEY,  
  gerüstnr  INTEGER NOT NULL,  
  kdnr      INTEGER NOT NULL,  
  datum     DATE NOT NULL,  
  dauer     INTEGER,  
  FOREIGN KEY (gerüstnr) REFERENCES gerüst(gerüstnr),  
  FOREIGN KEY (kdnr)      REFERENCES kunde(kdnr)  
);
```

SELECT-Reihenfolge

Klausel	Bedeutung
SELECT	Welche Spalten/Ausdrücke sollen im Ergebnis stehen?
FROM	Aus welchen Tabellen/Sichten kommen die Daten?
WHERE	Welche Zeilen werden gefiltert?
GROUP BY	Nach welchen Attributen wird gruppiert?
HAVING	Welche Gruppen werden gefiltert?
ORDER BY	Wie wird sortiert?

★ WHERE filtert einzelne Zeilen vor der Gruppierung. HAVING filtert Gruppen nach der Gruppierung. Das wird sehr gern abgefragt.

Views

Eine View ist eine logische Sicht auf vorhandene Tabellen. Sie speichert normalerweise nicht die Daten selbst, sondern die Beschreibung der SELECT-Anweisung. Views können wiederkehrende Abfragen vereinfachen, logische Datenunabhängigkeit unterstützen und Zugriffe einschränken.

- Einschränkungen bei änderbaren Views: keine berechneten Werte, keine GROUP BY-/DISTINCT-Klauseln, meist nur eine Basistabelle und alle NOT-NULL-Spalten müssen verfügbar sein.
- WITH CHECK OPTION kann verhindern, dass über die View Änderungen vorgenommen werden, die anschließend nicht mehr zur View gehören.

6. Datenbankentwurf mit ER-Modell

ER-Diagramm - Hochschulverwaltung (Chen-Notation)

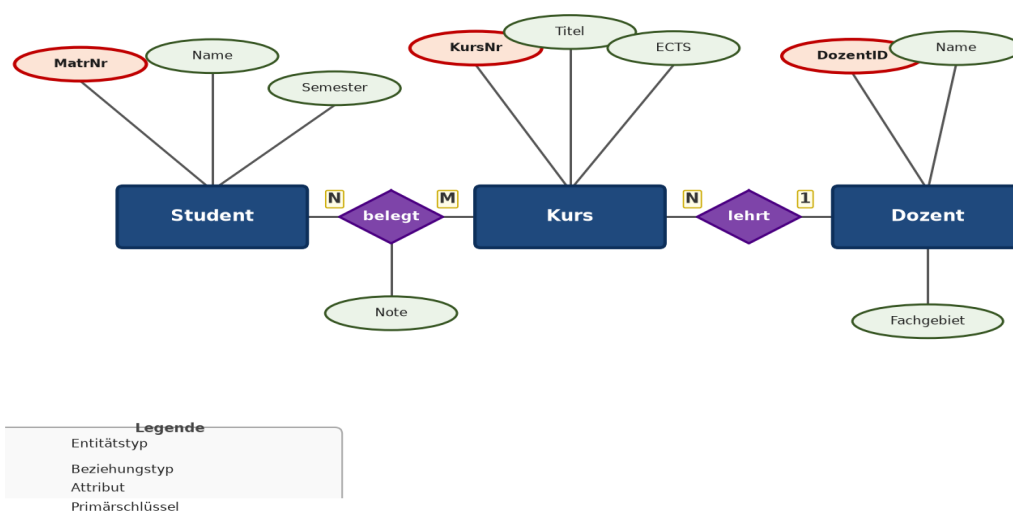


Abb.: ER-Diagramm Hochschule

Beim Datenbankentwurf wird aus einer verbalen Beschreibung der Realität ein Datenbankschema. Fehler im Entwurf sind später teuer, weil Anwendungen, Daten und Abfragen davon abhängen.

Anforderungen an den Entwurfsprozess

Anforderung	Bedeutung
Anwenderorientiertheit	Der Entwurf muss in der Begriffswelt der Nutzer formuliert und mit ihnen abgestimmt werden.
Determiniertheit	Für denselben Umweltausschnitt soll der Entwurfsprozess zu einem eindeutigen Modell führen.
Teilbarkeit	Ein Gesamtschema muss in Teilschemata zerlegbar sein.
Integrierbarkeit	Teilschemata müssen später wieder zu einem Gesamtschema zusammengeführt werden können.

Vorgehen aus einer Aufgabenstellung

1. **Objekte finden:** Substantive/Begriffe markieren, die eigene Daten besitzen: Student, Fach, Dozent.
2. **Attribute finden:** Welche Eigenschaften gehören zu welchem Objekt? Name, Geburtsdatum, Adresse.
3. **Beziehungen finden:** Verben/Beziehungswörter markieren: studiert in, absolviert, hält.
4. **Kardinalitäten bestimmen:** 1:1, 1:N, M:N sowie optional/obligatorisch.
5. **Schlüssel festlegen:** Primärschlüssel für Entitäten, Fremdschlüssel bei Beziehungen.
6. **Relationales Schema ableiten:** Entitäten werden Tabellen, M:N-Beziehungen werden Zwischentabellen.
7. **Normalisierung prüfen:** 1NF, 2NF, 3NF; Redundanzen und Anomalien vermeiden.

Kardinalitäten und Notationen

Notation	Bedeutung
(0,1)	Optional und höchstens ein zugeordnetes Objekt.
(1,1)	Pflichtbeziehung zu genau einem Objekt.
(0,*)	Optional mehrere Objekte möglich.
(1,*)	Mindestens ein, mehrere möglich.
Krähenfuß	Kreis = optional, Strich = obligatorisch, Krähenfuß = mehrfach.
Schwacher Entity-Typ	Kann nur zusammen mit einem identifizierenden starken Entity eindeutig existieren.

- ★ M:N-Beziehungen können im relationalen Modell nicht direkt als Fremdschlüssel in einer der beiden Tabellen gespeichert werden. Sie werden durch eine eigene Beziehungstabelle aufgelöst, die mindestens die Fremdschlüssel beider Seiten enthält.

7. Normalisierung

Normalisierung beschreibt Regeln für relationale Schemata. Ziel ist, Redundanzen und daraus entstehende Anomalien zu vermeiden. In der Praxis gelten die ersten drei Normalformen als besonders wichtig.

Anomalien

Anomalie	Bedeutung
Einfügeanomalie	Bestimmte Informationen können erst eingetragen werden, wenn andere eigentlich unabhängige Informationen vorhanden sind.
Löschanomalie	Beim Löschen eines Datensatzes gehen unbeabsichtigt weitere Informationen verloren.
Änderungsanomalie	Dieselbe Information muss an mehreren Stellen geändert werden; dabei entstehen leicht Widersprüche.

Funktionale Abhängigkeit

Eine funktionale Abhängigkeit $A \rightarrow B$ bedeutet: Aus dem Wert von A lässt sich eindeutig der zugehörige Wert von B bestimmen. Beispiel: Matrikelnummer $\rightarrow$ Name. Die Umkehrung muss nicht gelten, weil Namen mehrfach vorkommen können.

Die ersten drei Normalformen

Normalform	Regel	Typische Reparatur
1NF	Alle Attributwerte sind atomar; keine Listen oder wiederholten Gruppen in einem Feld.	Zusammengesetzte/mehrwertige Attribute aufteilen oder in eigene Tabelle auslagern.
2NF	1NF + jedes Nicht-Schlüsselattribut hängt vom gesamten Primärschlüssel ab.	Bei zusammengesetztem Schlüssel partielle Abhängigkeiten auslagern.
3NF	2NF + kein Nicht-Schlüsselattribut hängt von einem anderen Nicht-Schlüsselattribut ab.	Transitive Abhängigkeiten in eigene Tabellen zerlegen.

 1NF fragt: Ist jedes Feld atomar? 2NF fragt: Hängt alles vom ganzen Schlüssel ab? 3NF fragt: Hängt alles direkt vom Schlüssel ab und nicht von einem anderen Nicht-Schlüsselattribut?

Denormalisierung

Denormalisierung bedeutet, Normalformen bewusst zu verletzen, um Abfragen schneller zu machen. Das kann z. B. im Data-Warehouse sinnvoll sein, weil dort oft gelesen und wenig geändert wird. In normalen Anwendungssystemen muss man dabei auf Redundanzen und Anomalien achten.

8. Konsistenz, Integrität & Transaktionen

Konsistenz bedeutet, dass Modell, Daten und Operationen widerspruchsfrei zum abgebildeten Umweltausschnitt passen. Integritätsbedingungen geben dem DBMS Regeln, mit denen ungültige Zustände erkannt oder verhindert werden können.

Integritätsbedingung	Beispiel
Attributbezogen	CHECK (gehalt > 0), NOT NULL.
Tupelübergreifend	Gehalt eines Mitarbeiters muss kleiner als das des Direktors sein.
Mengenbezogen	Durchschnittsgehalt einer Gruppe muss in einem Bereich liegen.
Beziehungsbezogen	Fremdschlüsselwert muss in der referenzierten Tabelle existieren.

Transaktionen & ACID

Eine Transaktion ist eine logisch zusammengehörige Arbeitseinheit. Sie überführt eine Datenbank von einem konsistenten Zustand in einen anderen konsistenten Zustand.

ACID	Bedeutung
A – Atomarität	Alles oder nichts: Scheitert ein Teil, werden bisherige Änderungen zurückgesetzt.
C – Consistency/Konsistenz	Eine erfolgreiche Transaktion hinterlässt wieder einen konsistenten Zustand.
I – Isolation	Parallele Transaktionen sollen sich nicht durch Zwischenzustände beeinflussen.
D – Durability/Dauerhaftigkeit	Nach COMMIT bleiben Änderungen auch nach Systemfehlern erhalten.

Transaktionstypen

Typ	Bedeutung
Autocommit	Jede SQL-Anweisung ist automatisch eine eigene Transaktion.
Implizite Transaktion	Eine neue Transaktion startet automatisch, muss aber mit COMMIT/ROLLBACK beendet werden.
Explizite Transaktion	BEGIN/START TRANSACTION startet bewusst eine Transaktionsklammer.

```
BEGIN TRANSACTION;
UPDATE konto SET saldo = saldo - 100 WHERE konto_id = 1;
UPDATE konto SET saldo = saldo + 100 WHERE konto_id = 2;
COMMIT;          -- beide Änderungen dauerhaft
-- oder bei Fehler: ROLLBACK;
```

9. Recovery-Verfahren

Recovery ist die Wiederherstellung eines möglichst aktuellen konsistenten Datenbankzustands nach einem Fehler. Voraussetzung sind Sicherungen der Datenbankinhalte und Log-Informationen über Änderungen.

Logging

Log-Art	Inhalt / Zweck
Physisches Log	Informationen über konkrete Datenbankinhalte oder Blöcke.
Logisches Log	Informationen über Operationen bzw. Änderungen auf logischer Ebene.
Zustandslog	Vorher-/Nachher-Zustände von Daten.
Übergangslog	Beschreibung der durchzuführenden Änderung.

Rollback und Rollforward

Verfahren	Idee
Rollback	Nicht abgeschlossene oder fehlerhafte Transaktionen werden rückgängig gemacht.
Rollforward	Ausgehend von einer Sicherung werden protokollierte Änderungen erneut angewendet.

Grober Recovery-Ablauf

8. Fehler erkennen und DBMS in kontrollierten Wiederanlauf bringen.
9. Letzte konsistente Sicherung bzw. letzten Checkpoint bestimmen.
10. Nicht abgeschlossene Transaktionen mit Undo/Rollback zurücksetzen.
11. Bereits bestätigte Transaktionen mit Redo/Rollforward nachvollziehen.
12. Datenbank wieder für den normalen Betrieb freigeben.

 Nenne immer Ziel, Voraussetzung und Ablauf: Ziel = konsistenter aktueller Zustand; Voraussetzung = Sicherung + Log; Ablauf = Undo nicht abgeschlossener und Redo bestätigter Transaktionen.

10. Sperren, Isolation & Synchronisation

Bei Mehrbenutzerbetrieb greifen mehrere Transaktionen gleichzeitig auf dieselben Daten zu. Ohne Synchronisation können falsche Ergebnisse oder inkonsistente Zustände entstehen.

Typische Nebenläufigkeitsprobleme

Problem	Beschreibung
Lost Update	Zwei Transaktionen ändern denselben alten Wert; eine Änderung überschreibt die andere.
Dirty Read	Eine Transaktion liest Daten, die von einer anderen noch nicht committed wurden.
Non Repeatable Read	Dasselbe Tupel liefert bei erneutem Lesen andere Werte.
Phantom Read	Eine zweite Abfrage mit gleichem Prädikat findet zusätzliche oder fehlende Tupel.

Sperren

Sperrart	Bedeutung
Shared Lock / Lesesperre	Mehrere Transaktionen dürfen gleichzeitig lesen.
Exclusive Lock / Schreibsperre	Zum Ändern; andere Lese-/Schreibzugriffe werden je nach Verfahren blockiert.
Sperrgranulat	Größe des gesperrten Bereichs: Satz, Block, Tabelle, Indexbereich.
Prädikatsperre	Sperrt einen Wertebereich bzw. ein Suchprädikat, um Phantome zu vermeiden.

Isolationsebenen nach SQL

Isolationsebene	Verhindert sicher	Kann noch zulassen
READ UNCOMMITTED	praktisch nichts	Dirty Reads, Non Repeatable Reads, Phantoms
READ COMMITTED	Dirty Reads	Non Repeatable Reads, Phantoms
REPEATABLE READ	Dirty Reads, Non Repeatable Reads	Phantoms
SERIALIZABLE	Dirty Reads, Non Repeatable Reads, Phantoms	höchster Aufwand/geringste Parallelität

Deadlocks und optimistische Synchronisation

- Deadlock: Transaktionen warten zyklisch aufeinander. Gegenmaßnahmen: Abbruch einer Transaktion, Timeout, Deadlock-Erkennung.
- Optimistische Synchronisation: Es wird zunächst nicht gesperrt. Am Ende wird validiert, ob Konflikte aufgetreten sind; bei Konflikt wird zurückgesetzt.
- 2-Phasen-Commit: Bei verteilten Transaktionen fragt ein Koordinator alle Teilnehmer, ob sie bereit sind. Nur wenn alle READY melden, wird global committed.



Je höher die Isolation, desto weniger Nebenläufigkeitsfehler, aber desto mehr Sperren/Wartezeiten. Das ist der zentrale Zielkonflikt.

11. Datenspeicherung & Pufferverwaltung

DBMS müssen externe Speicherzugriffe minimieren, weil zwischen Hauptspeicher und Externspeicher eine große Zugriffslücke besteht. Deshalb spielen Blöcke, Puffer, Indexe und Speicherorganisation eine zentrale Rolle.

Speicherhierarchie

Ebene	Eigenschaft
Register / Cache	Sehr schnell, sehr klein, CPU-nah.
Hauptspeicher	Schneller als Festplatte, flüchtig.
Externspeicher online	Festplatten/SSDs, blockadressierbar, dauerhaft.
Archivspeicher	Langsamer, für Sicherungen und seltene Daten.

DBMS-Puffer

- DBMS nutzen eigene Puffer, weil Betriebssystempuffer nicht auf typische Datenbankzugriffe optimiert sind.
- Sequenzielle Scans, Indexzugriffe und Log-Schreibvorgänge haben unterschiedliche Zugriffsmuster.
- Ersetzungsstrategien entscheiden, welcher Pufferrahmen überschrieben wird: LRU, LFU, LRU-K, Clock-Algorithmus.

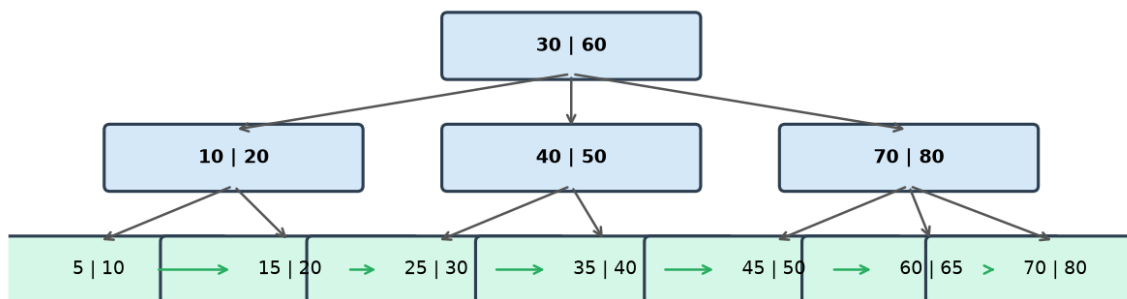
Speicherorganisation

Begriff	Bedeutung
Block	Einheit, die typischerweise in einem I/O-Vorgang gelesen/geschrieben wird.
Segment	Speicherbereich für ein Datenbankobjekt, z. B. Tabelle oder Index.
Extent	Zusammenhängender Speicherbereich innerhalb eines Segments.
Table Space	Container, der Dateien und Datenbankobjekte umfasst.
Heap	Tupel werden ohne Sortierung eingefügt; einfache Speicherung, aber Suche oft teuer.
Partition	Horizontale Aufteilung sehr großer Tabellen in disjunkte Teile.
Cluster	Physisch nahe Speicherung logisch zusammengehöriger Tupel, oft zur Join-Unterstützung.

 Performance bei Datenbanken heißt oft: weniger physische Blockzugriffe. Indexe, Puffer und Clustering sind Mittel, um diese Blockzugriffe zu reduzieren.

12. Indexe, B-Bäume & Zugriffspfade

B+-Baum - Beispiel mit Schlüsseln



Grün = Blattknoten (alle Datensätze, verkettet) | Blau = innere Knoten (nur Schlüssel)

Abb.: B+-Baum Beispiel

Ein Index ist ein zusätzlicher Zugriffspfad: Er ordnet Suchschlüsselwerte den Speicherorten passender Datensätze zu. Dadurch werden Punkt- und Bereichsanfragen schneller, aber INSERT/UPDATE/DELETE werden aufwendiger, weil der Index gepflegt werden muss.

Index-/Speicherform	Eigenschaft
Non-Clustered Index	Index und Daten liegen getrennt; Blattebene verweist auf Datensätze.
Clustered Index / IOT	Daten sind nach Indexschlüssel sortiert bzw. in der Blattebene gespeichert.
B+-Baum	Innere Knoten führen die Suche; Datensätze/Verweise liegen auf Blattebene; Blätter sind oft verkettet.
Hash-Zugriff	Gut für Gleichheitsabfragen, ungeeignet für Bereichsanfragen.

Grid File	Mehrdimensionaler Zugriffspfad, z. B. für räumliche Punkt- oder Gebietsanfragen.
-----------	--

B-Baum/B+-Baum – wichtige Idee

- Knoten enthalten mehrere sortierte Schlüssel und Verweise.
- Alle Blätter liegen auf gleicher Tiefe; dadurch bleiben Suchkosten stabil.
- Beim Einfügen in einen vollen Knoten wird gesplittet und ein Schlüssel nach oben gezogen.
- Beim Löschen kann Unterlauf entstehen; dann wird mit Nachbarknoten umverteilt oder zusammengeführt.

! Bei den Übungsunterlagen wird B-Baum 2. Ordnung so verwendet: maximal $2 \cdot 2 = 4$ Schlüssel pro Knoten. Beim 5. Schlüssel muss gesplittet werden. Genau diese Regel solltest du in der mündlichen Prüfung zuerst nennen.

Beispiel aus der Übung

Einfügefølge: 1, 3, 5, 7, 12, 21, 36, 84, 9, 10, 15, 22. Bei Ordnung 2 ergibt sich am Ende:

```
      [ 5 | 10 | 21 ]
     /      |      \
[1|3] [7|9] [12|15] [22|36|84]
```

Typische mündliche Fragen

- **Frage:** Warum braucht man Indexe?
- **Antwortidee:** Zur Beschleunigung von Such- und Join-Operationen, besonders bei großen Tabellen.
- **Frage:** Was ist der Nachteil eines Index?
- **Antwortidee:** Zusätzlicher Speicherbedarf und zusätzlicher Pflegeaufwand bei Änderungen.

13. Anfrageverarbeitung & Optimierung

Anfrageverarbeitung übersetzt eine SQL-Anfrage in ausführbare Operationen. Anfrageoptimierung versucht, einen möglichst günstigen Ausführungsplan zu finden, weil dieselbe logische Anfrage auf viele physische Arten abgearbeitet werden kann.

Grobablauf

13. SQL-Anweisung syntaktisch und semantisch prüfen.
14. In eine interne Darstellung/Relationenalgebra übersetzen.
15. Logische Umformungen durchführen, z. B. Selektionen früh nach unten schieben.
16. Alternative Ausführungspläne erzeugen: Join-Reihenfolgen, Indexnutzung, Table Scan usw.
17. Kosten abschätzen und den günstigsten Plan auswählen.
18. Plan ausführen und Ergebnis liefern.

Regelbasierte vs. kostenbasierte Optimierung

Methode	Idee	Benötigt
Regelbasiert	Nutzen fester Regeln der Relationenalgebra, z. B. Projektion/Selektion möglichst früh.	Regelsatz, keine detaillierten Statistiken.
Kostenbasiert	Vergleicht mögliche Pläne anhand geschätzter Kosten.	Aktuelle Statistiken, Selektivitäten, Indexinformationen.

Einflussmöglichkeiten

- Geeignete Indexe anlegen oder entfernen.
- Statistiken aktuell halten, damit Selektivitäten realistisch geschätzt werden.
- Abfragen so formulieren, dass Indexe nutzbar bleiben.
- Datenmodell, Partitionierung oder Clustering an typische Workloads anpassen.
- In manchen DBMS Optimizer-Hints verwenden, aber nur begründet.

14. Benutzerverwaltung & Rechte

Die Benutzerverwaltung regelt, wer sich mit dem DBMS verbinden darf und welche Datenbankobjekte oder SQL-Anweisungen verwendet werden dürfen. Ziel ist Zugriffskontrolle und Datenschutz.

Begriff	Bedeutung
Benutzerkonto	Identität zur Anmeldung am Datenbanksystem.
Profil	Limitierungen für Ressourcen, z. B. Sitzungen, CPU-Zeit, Passwortlaufzeit.
Rolle	Benannte Menge von Rechten; wird Benutzern zugewiesen.
Objektrecht	Recht auf Datenbankobjekt, z. B. SELECT auf Tabelle personal.
Systemprivileg	Recht zur Nutzung bestimmter SQL-Anweisungen, z. B. CREATE TABLE.
GRANT / REVOKE	Rechte vergeben bzw. entziehen.

```
CREATE ROLE normaler_nutzer;  
GRANT SELECT ON personal TO normaler_nutzer;  
GRANT normaler_nutzer TO schulze;  
REVOKE SELECT ON personal FROM normaler_nutzer;
```

- ★ Rollen sind in größeren Organisationen besser als Einzelrechte, weil Rechte nach Aufgaben organisiert werden. Bei Aufgabenwechsel muss dann nur die Rolle geändert werden.

15. Prüfungsaufgaben aus den Unterlagen

Die Klausur- und Übungsunterlagen zeigen sehr deutlich, welche Themen prüfungsnah sind: SQL-Constraints, B-Baum-Einfügen, ER-Modellierung mit relationalem Schema, Recovery und Anfrageoptimierung.

Aufgabe: CREATE TABLE mit Constraints

```
CREATE TABLE kunde (  
  kdnr      INTEGER PRIMARY KEY,  
  name      VARCHAR(30) NOT NULL,  
  vorname   VARCHAR(30),  
  adresse   VARCHAR(60)  
);  
  
CREATE TABLE gerüst (  
  gerüstnr  INTEGER PRIMARY KEY,
```

```

    typ          CHAR(30) NOT NULL,
    Rüstfläche   INTEGER CHECK (Rüstfläche > 0),
    mietpreis    NUMERIC(10,2) CHECK (mietpreis > 0)
);

CREATE TABLE miete (
    mvnr         INTEGER PRIMARY KEY,
    gerüstnr     INTEGER NOT NULL,
    kdnr         INTEGER NOT NULL,
    datum       DATE NOT NULL,
    dauer        INTEGER,
    FOREIGN KEY (gerüstnr) REFERENCES gerüst(gerüstnr),
    FOREIGN KEY (kdnr) REFERENCES kunde(kdnr)
);

```

! Achte auf die Punkte: Primärschlüssel kennzeichnen, NOT NULL für Zwangsfelder setzen, CHECK-Bedingungen für 'größer 0' verwenden und Fremdschlüssel syntaktisch sauber angeben.

Aufgabe: Hochschule als ER-Modell

Objektklasse / Relation	Attribute / Schlüsselidee
Student	Matrikelnummer (PK), Vorname, Nachname, Geburtsdatum, Straße, Ort, PLZ, Hausnr.
Studienrichtung	Bezeichnung (PK), Abschluss, SBID (FK).
Fach	FachID (PK), Name, Stundenumfang.
Studienbereich	SBID (PK), Name, Beschreibung.
Dozent	Personalnummer (PK), Vorname, Nachname, Titel, Geburtsdatum, Adresse, SBID (FK).
studiert_in	Matrikelnummer (FK), Bezeichnung (FK), Beginn, Ende, Abschlussnote.
absolviert	Matrikelnummer (FK), FachID (FK), Prüfung1, Prüfung2.
besteht_aus	Bezeichnung (FK), FachID (FK).
hält	FachID (FK), Personalnummer (FK).

16. Kurz-Wiederholung für die mündliche Prüfung

Diese Seite ist als schnelle Wiederholung vor der Prüfung gedacht. Sprich die Antworten am besten laut aus und prüfe, ob du die Begriffe ohne Ablesen erklären kannst.

Thema	Das musst du sagen können
Grundidee DB	Zentraler, redundanzarmer Datenbestand unter Kontrolle eines DBMS.
ANSI-SPARC	Externe, konzeptuelle, interne Ebene; logische und physische Datenunabhängigkeit.
Relationales Modell	Relation, Tupel, Attribut, Domäne, Primärschlüssel, Fremdschlüssel.
SQL	DDL/DML/DCL/TCL; CREATE TABLE mit PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK.

ER-Modell	Objekte, Attribute, Beziehungen, Kardinalitäten; M:N wird Zwischentabelle.
Normalformen	1NF atomar; 2NF voller Schlüssel; 3NF keine transitive Abhängigkeit.
Transaktion	ACID, COMMIT, ROLLBACK, konsistenter Zustand.
Recovery	Sicherung + Log, Undo/Rollback und Redo/Rollforward.
Isolation	Dirty Read, Non Repeatable Read, Phantom Read; SERIALIZABLE am stärksten.
Index	Beschleunigt Lesen/Suchen, kostet Speicher und Pflege bei Änderungen.
Optimierung	Ausführungsplan, regel-/kostenbasiert, Statistiken und Indexe.
Rechte	Benutzer, Rollen, GRANT, REVOKE, Objektrechte, Systemprivilegien.



In der mündlichen Prüfung ist eine klare Struktur oft wichtiger als perfekte Formulierungen. Starte mit einer Definition, nenne dann die wichtigsten Unterpunkte und gib zum Schluss ein kurzes Beispiel.

Viel Erfolg bei der mündlichen Prüfung! 🎓